

Praan AI/ML Intern Task: 3D Point Cloud Reconstruction

1. Objective

The goal of this task is to take a 30–60 second handheld video of a room or object and convert it into a 3D point cloud.

Note on Model Building vs. Time Constraints: Building and training a new depth estimation or 3D reconstruction model from scratch requires massive datasets (like ScanNet or NYU Depth) and weeks of GPU training time. Due to the limited time frame for this task, I focused on building a smart, custom pipeline using state-of-the-art pre-trained computer vision models to achieve high-quality results efficiently.

2. General Pipeline Overview

Instead of processing every single frame individually (which causes overlapping mess and scaling issues), I built a pipeline that works like this:

- 1. Frame Extraction:** Used ffmpeg to extract frames from the video at 2 frames per second.
- 2. Panorama Stitching:** Used OpenCV to stitch these frames into one single, wide 2D panorama image. This solves the issue of aligning camera angles.
- 3. Depth Estimation:** Used AI models to predict the depth of every pixel in the panorama.
- 4. 3D Projection:** Used math (trigonometry) to project the 2D image and depth map into 3D space (X, Y, Z coordinates).
- 5. Noise Cleanup:** Used scipy (cKDTree) to remove floating point errors and statistical outliers.

To explore the best possible result, I built and tested two different approaches.

3. Approach 1: Relative Depth (Depth Anything V2)

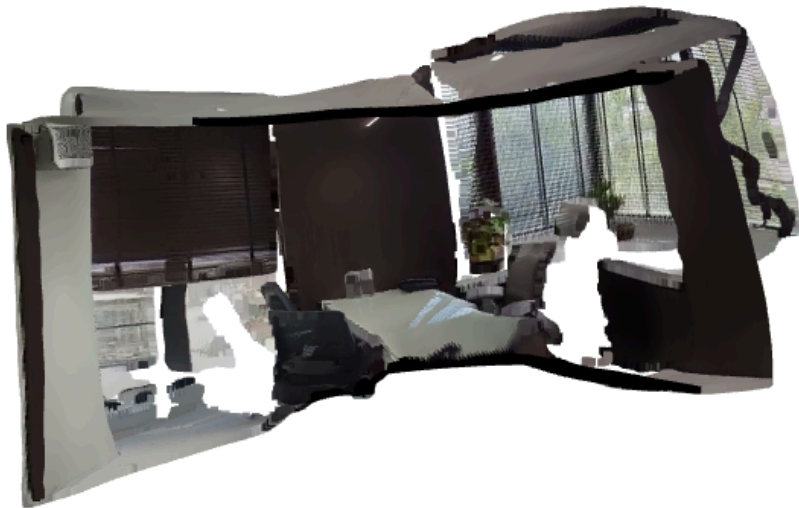
Tools used: HuggingFace transformers, Depth-Anything-V2-Small, OpenCV

In this approach, I used Depth Anything V2, a very fast and lightweight model.

- How it works: It predicts "relative depth," meaning it knows what is in front and what is in the back, but it doesn't know the exact measurements in real-world meters.

- Projection: Because the output is scaleless, I scaled it mathematically and used a Cylindrical Projection to bend the flat panorama into a 3D room.

Output for Approach 1



4. Approach 2: Metric Depth (ZoeDepth)

Tools used: PyTorch Hub, ZoeDepth (ZoeD_N), OpenCV, Plotly, plyfile

In this approach, I used ZoeDepth, a model specifically trained on indoor spaces (NYU Depth V2 dataset) to predict exact real-world depth.

- How it works: It predicts "metric depth," meaning the output values are actual distances in meters.
- Projection: Because I had real distances, I used a full Spherical Projection. This creates a much better shape for the room, especially for the floor and ceiling.

Output for Approach 2



5. Comparison: Which is better?

Feature	Approach1(DepthAnythingV2)	Approach2(ZeoDepth)
Speed & size	Very fast, lightweight model	Slower, heavier Model(~1.4GB)
Scale	Relative (guessed scale)	Metric (Real-world meters)
Object Edges	Extremely sharp and detailed	Slightly softer edges
Best for	General scenes, outdoors, or when processing speed is top priority	Indoor rooms where actual measurements are needed.

Conclusion on which to use:

- If Praan needs a pipeline to scan indoor rooms to get real-world measurements, Approach 2 is the clear winner.
- If Praan needs to scan unpredictable environments (like outdoor spaces or random objects) quickly on edge devices, Approach 1 is better because it generalizes to anything without needing specific indoor training data.

6. Known Limitations & Future Improvements

My current pipeline relies on OpenCV's 2D panorama stitcher. This works great if the person holding the camera stood in one place and just rotated their body. However, if the person walked forward (translation), 2D stitching can fail due to parallax.

Next Steps: If I were to improve this during the internship, I would replace the 2D stitching step with a true 3D Structure-from-Motion (SfM) tool like COLMAP or a modern AI like DUST3R. This would track the exact 3D movement of the camera through the room, allowing us to process videos where the user is walking around freely.